

WAVESPARKS COMMUNITY

# WaveSparks Community Admin Operations Manual

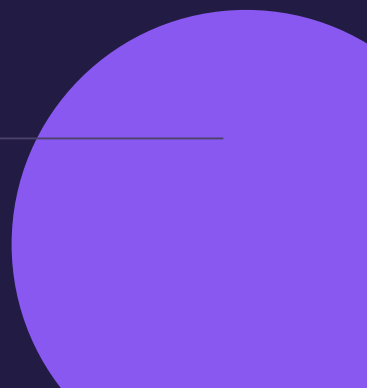
Community administration, governance, and infrastructure maintenance

---

Audience: Admin

Version: 1.0

Updated: 2026-07-29



# Contents

---

|                                                                                                    |    |
|----------------------------------------------------------------------------------------------------|----|
| 1. Admin responsibilities and the complete operating lifecycle . . . . .                           | 3  |
| 2. Known risks that must be resolved before launch . . . . .                                       | 4  |
| 2.1 P0: The first Admin cannot complete identity connection in a new production database . . . . . | 4  |
| 2.2 P1: Some administrative Route Handlers still authorize by email . . . . .                      | 5  |
| 2.3 P1: Preview Accounts API must not be part of a production operating path . . . . .             | 5  |
| 2.4 P1/P2: Controls not yet implemented . . . . .                                                  | 5  |
| 3. Authorization and responsibility boundaries . . . . .                                           | 5  |
| 4. Admin pages and responsibilities . . . . .                                                      | 6  |
| 5. Member and invitation lifecycle . . . . .                                                       | 6  |
| 5.1 Invite one person . . . . .                                                                    | 6  |
| 5.2 Bulk import . . . . .                                                                          | 7  |
| 5.3 Account states and security actions . . . . .                                                  | 8  |
| 5.4 Mentor designation . . . . .                                                                   | 8  |
| 6. Community and Event lifecycle . . . . .                                                         | 8  |
| 6.1 Create and publish an Event . . . . .                                                          | 9  |
| 6.2 Add to Main Community . . . . .                                                                | 9  |
| 7. Profile, content, Introduction, and Matching operations . . . . .                               | 10 |
| 7.1 Profiles . . . . .                                                                             | 10 |
| 7.2 Content moderation . . . . .                                                                   | 10 |
| 7.3 Introductions . . . . .                                                                        | 10 |
| 7.4 Matching . . . . .                                                                             | 10 |
| 8. System architecture and systems of record . . . . .                                             | 12 |
| 9. Environment variable management . . . . .                                                       | 12 |
| 10. Release and database migration runbook . . . . .                                               | 13 |
| 10.1 Before release . . . . .                                                                      | 13 |
| 10.2 Migration . . . . .                                                                           | 13 |
| 10.3 Deployment and verification . . . . .                                                         | 14 |
| 11. Vercel operations and maintenance . . . . .                                                    | 14 |
| 11.1 Current responsibilities . . . . .                                                            | 14 |
| 11.2 Daily checks . . . . .                                                                        | 14 |
| 11.3 Environment variables and secret rotation . . . . .                                           | 14 |
| 11.4 Rollback . . . . .                                                                            | 15 |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>12. Clerk operations and maintenance</b>          | <b>15</b> |
| 12.1 Current boundary                                | 15        |
| 12.2 Initial configuration                           | 16        |
| 12.3 Daily operations and incident response          | 16        |
| 12.4 Zero-downtime rotation                          | 16        |
| <b>13. Neon operations and maintenance</b>           | <b>17</b> |
| 13.1 Current boundary                                | 17        |
| 13.2 Connection selection                            | 17        |
| 13.3 Migration, branching, and recovery              | 17        |
| 13.4 Daily monitoring                                | 17        |
| 13.5 Credential rotation                             | 18        |
| <b>14. Resend operations and maintenance</b>         | <b>18</b> |
| 14.1 Current boundary                                | 18        |
| 14.2 Domain and API Key                              | 18        |
| 14.3 Daily checks and response                       | 18        |
| 14.4 Zero-downtime rotation                          | 19        |
| <b>15. Vercel Blob and media maintenance</b>         | <b>19</b> |
| <b>16. Security incident and offboarding runbook</b> | <b>19</b> |
| 16.1 Member account incident                         | 19        |
| 16.2 Administrator departure                         | 20        |
| 16.3 User deletion                                   | 20        |
| <b>17. Recurring maintenance checklist</b>           | <b>20</b> |
| Daily                                                | 20        |
| Weekly                                               | 20        |
| Monthly                                              | 20        |
| Quarterly                                            | 21        |
| <b>18. Troubleshooting quick reference</b>           | <b>21</b> |
| <b>19. Release acceptance checklist</b>              | <b>21</b> |

Version: 1.0 Audit baseline: codebase, current interface, and official vendor documentation as of July 29, 2026

Intended audience: WaveSparks Community organization administrators, release owners, and frontline operators

This manual distinguishes between behavior implemented in the current codebase, actions performed in vendor consoles, and controls that must be fixed or established before launch. Before making a production change, confirm the target environment, recovery point, and authorized approver. Never paste secrets into tickets, chat messages, or screenshots.

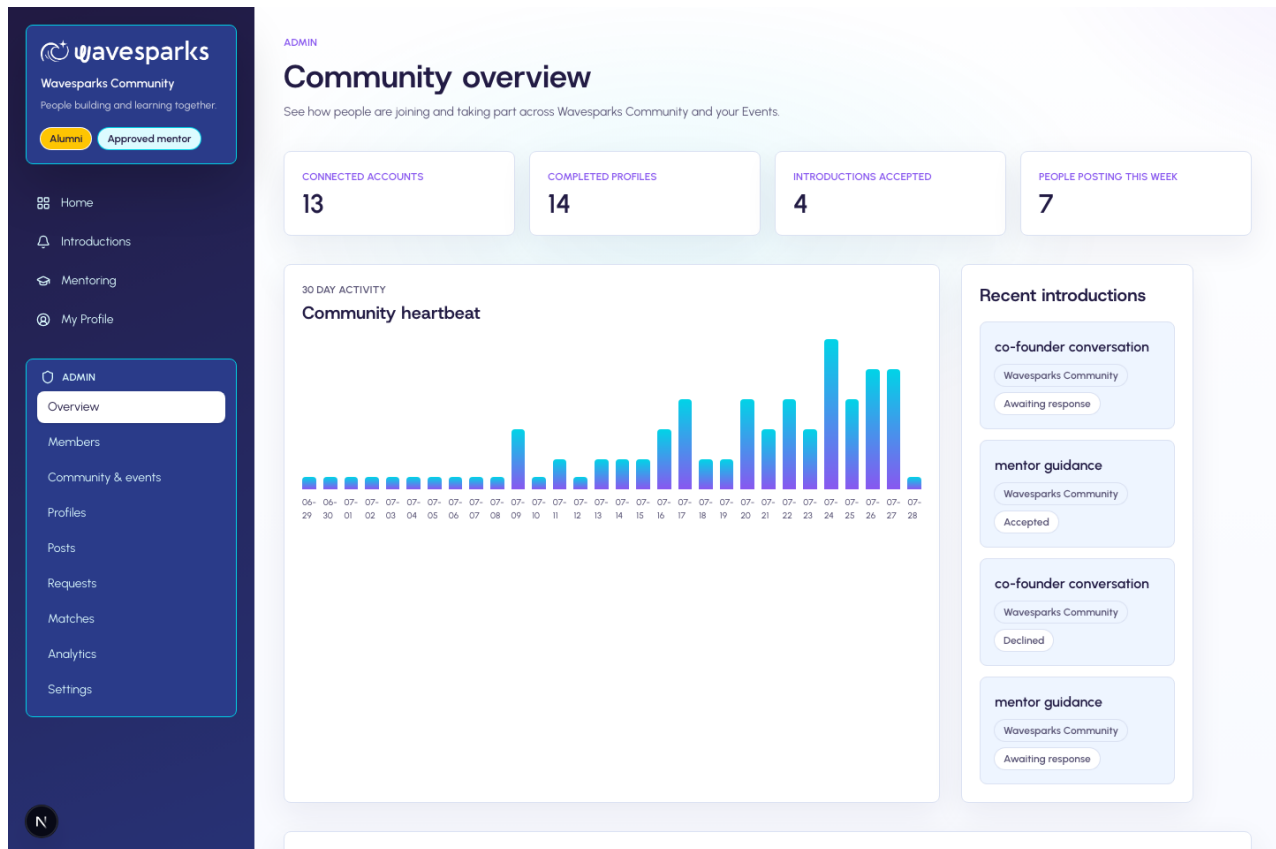
## 1. Admin responsibilities and the complete operating lifecycle

Admins manage community accounts, Spaces, content, introductions, matching, and day-to-day operations.

Vercel, Clerk, Neon, and Resend provide deployment, identity, data, and product-email services respectively.

The standard operating lifecycle is:

1. Resolve the first-Admin identity connection blocker and establish an auditable production administrator.
2. Configure the production domain, environment variables, Clerk webhook, Neon database, Resend domain, and Vercel Blob.
3. Run environment audits, migrations, data audits, tests, and deployment verification.
4. Configure the community name, logo, description, and invitation guidance.
5. Create Events and configure lifecycle state, timing, participants, and Matching.
6. Invite individual members or import them in bulk, then follow invitation acceptance through completion.
7. Manage account permission, Mentor designation, account status, and per-Space access as independent dimensions.
8. Operate Profiles, posts, Introductions, Matching, and Analytics.
9. Handle account suspension, content moderation, security incidents, and data anonymization.
10. Check deployments, Cron, email, and webhooks daily; rotate secrets, rehearse recovery, and review capacity regularly.



Admin Overview

## 2. Known risks that must be resolved before launch

The items below are not optional documentation notes. They are production risks confirmed by the current code audit.

### 2.1 P0: The first Admin cannot complete identity connection in a new production database

`scripts/bootstrap-production.ts` creates an `org_admin` membership but does not explicitly set `accountStatus`, so the invited default is used. The script also does not create a local one-time Invitation, send a Clerk Application Invitation, or write a `clerkUserId`.

Production authentication connects local identity only by Clerk User ID and explicitly forbids automatic claiming by email. Therefore, after `db:bootstrap` runs against a new database, the first Admin cannot enter the Admin interface merely by signing in to Clerk with the same email address.

Release requirements:

- Implement a supported first-Admin invitation and binding flow in code, with automated tests.
- Do not temporarily enable automatic identity binding by email.
- Do not manually mark a database record as connected without binding its Clerk User ID.
- Do not use Preview Accounts as production administrators.
- Do not declare first-Admin provisioning complete until the fix is merged, migrated, rehearsed, and reviewed by a second person.

## 2.2 P1: Some administrative Route Handlers still authorize by email

The following endpoints conflict with the Clerk-ID identity invariant and should use the server-side Viewer Context and Clerk ID before release:

- Profile CSV export;
- Organization logo upload;
- Member import parsing;
- Interactive Matching recompute;
- Preview Accounts provisioning.

The Profile CSV response should also add `Cache-Control: no-store` and an export audit record. Until these issues are fixed, restrict Admin access, do not share Admin accounts, and inspect logs and data outcomes after every sensitive operation.

## 2.3 P1: Preview Accounts API must not be part of a production operating path

`POST /api/internal/preview-accounts` currently has no explicit production or E2E guard. It can write four connected QA accounts, including an Admin account. This is not a production tool. Disable or remove it before launch, or add a strict non-production guard.

## 2.4 P1/P2: Controls not yet implemented

- Although the schema contains `reports` and `admin_actions`, the product currently has neither a report queue nor a central Admin Audit Log.
- Suspending a user in the application does not automatically revoke Clerk Sessions. A credential incident requires a separate Clerk action.
- There is no GitHub Actions release gate, Sentry/OpenTelemetry integration, Log Drain, or alerting-as-code.
- The Neon recovery process and drill records are not automated; runtime and migration processes share one database credential.
- Resend has no webhook, delivery-status table, or Admin retry interface.

This manual provides manual control procedures for these gaps. Do not describe any of them as automated controls.

# 3. Authorization and responsibility boundaries

Admin access requires a `connected` account with either `org_admin` or `platform_owner`. There are currently no fine-grained roles such as "member administrator" or "content moderator"; both Admin roles have full administrative privileges.

Authorization is divided into independent dimensions:

- Account permission: Member or Administrator;
- Mentor designation: `not_mentor`, `needs_review`, or `approved`;
- Account status: `invited`, `connected`, `suspended`, or `deprovisioned`;
- Space entitlement: `active`, `waitlist`, `rejected`, `suspended`, or `removed`;
- Space lifecycle: `draft`, `upcoming`, `active`, `ended`, or `archived`.

An Approved Mentor does not automatically receive Admin access. An Admin can audit every Space, but must be explicitly added to a Space to appear as a social participant in People, publish content, participate in matching, or create a manual Introduction from that Space.

An Admin cannot revoke their own effective Admin permission. Require a second confirmation when changing another administrator. Maintain at least two independent administrators protected by MFA to avoid a single point of operational failure.

## 4. Admin pages and responsibilities

| Page               | Primary capabilities                                                                                      | Key considerations                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Overview           | Connected accounts, complete Profiles, accepted Introductions, weekly posters, and recent activity        | This is an operating summary, not a monitoring or alerting system    |
| Members            | Individual invitations, CSV/XLSX import, invitation retry/revoke, role, Mentor, account, and Space status | Evaluate all four authorization dimensions independently             |
| Community & events | Main Community, Event create/update/archive/restore, Participants, Content, and Matching                  | ended remains interactive; only archived stops access                |
| Profiles           | View complete Profiles and contact details, Featured/Stale controls, and CSV export                       | Contains sensitive data; central export auditing is currently absent |
| Posts              | Hide/Unhide, Feature, Lock comments, Archive, and image/link/comment moderation                           | There is currently no user report entry point                        |
| Requests           | Review by status, source, or Space; create a manual Introduction                                          | The Admin must have access to the source Space                       |
| Matches            | Recompute, configure match type/direction/weights/minimum score, review anonymous feedback and runs       | Inspect results and feedback after configuration changes             |
| Analytics          | Account, Profile, Introduction, teams-formed, and activity trends                                         | This is currently a basic snapshot, not a complete BI system         |
| Settings           | Name, logo, tagline, community description, and invitation guidance                                       | The logo is stored in a public Blob                                  |

## 5. Member and invitation lifecycle

### 5.1 Invite one person

1. Open Members -> Invite people -> One person.
2. Enter the exact email address and an optional name.
3. Select Member or Administrator.
4. Independently select Not a mentor or Approved mentor. An invitation cannot create `needs_review`.
5. For a Member, select the target Main Community or Event and the initial Space access state.
6. An Administrator can be invited without a Space; add one only if the Admin needs to participate socially.
7. After submission, inspect the row-level Invitation status. Do not treat Clerk's acceptance of the send request as proof of delivery.

WaveSparks first creates a local one-time invitation, stores only its token hash in Neon, and then asks Clerk to send an Application Invitation. Clerk creates or signs in the identity; it does not create a Clerk Organization, role,

or community permission. The recipient must accept within seven days using the exact matching verified emailaddress.

Invitation statuses:

- **pending**: valid and awaiting acceptance;
- **accepted**: local identity connection completed;
- **revoked**: invitation revoked;
- **expired**: validity period elapsed;
- **failed**: local creation or Clerk delivery request failed.

Resending creates a new valid ticket. After revocation, the old link can no longer be used. Investigate invitation email problems in Clerk, not Resend.

## 5.2 Bulk import

The importer accepts **.csv**, **.xlsx**, or pasted CSV data:

1. Upload or paste the data.
2. Map the required Email field and optional Name field.
3. Select exactly one target Space and initial access state.
4. Correct or remove invalid rows in Preview.
5. Confirm Invite N people.
6. Review row-level outcomes and retry only failed rows.

Limits: 2 MB, first worksheet only, no more than 20 columns, and no more than 100 non-empty data rows.

Selecting a file does not send invitations. The file is parsed in memory and not retained. A bulk import always creates Member + Not a mentor; it cannot grant Admin access or Mentor approval in bulk.

Existing Profiles and global account permissions are not overwritten. Import only adds the target Space entitlement when it is safe to do so. For a repeated email address, the first row wins. Invalid rows do not block valid rows.



**Members**

Search, invite, and manage accounts, including access to Wavesparks Community and Events.

14 members

Account permissions, mentor designation, and community or Event access are tracked separately.

| Name or email                                                                             | Account status | Invitation | Mentor designation | Community or Event            |
|-------------------------------------------------------------------------------------------|----------------|------------|--------------------|-------------------------------|
| Priya<br>priya@example.com<br>Invited outsider offering brand and storytelling mentorship | Active         | Connected  | Mentor review      | None                          |
| Tom<br>tom@example.com<br>Founder-operator with logistics scars and strong B2B instincts  | Active         | Connected  | Approved mentor    | Wavesparks Community - Active |

Members management

### 5.3 Account states and security actions

- invited: identity is not connected and cannot enter any Space.
- connected: the user can operate according to each Space entitlement.
- suspended: reversible global security block that overrides every Space.
- deprovisioned: the organization no longer provisions the account; every Space is blocked.

Use Space-level **suspended** or **removed** when only one Event is affected. Use account suspension or deprovisioning for credential compromise, serious violations, or organization-wide departure.

During a security incident, application suspension blocks application authorization but does not revoke existing Clerk Sessions. Also revoke Sessions or block the identity in Clerk, and rotate secrets when appropriate.

### 5.4 Mentor designation

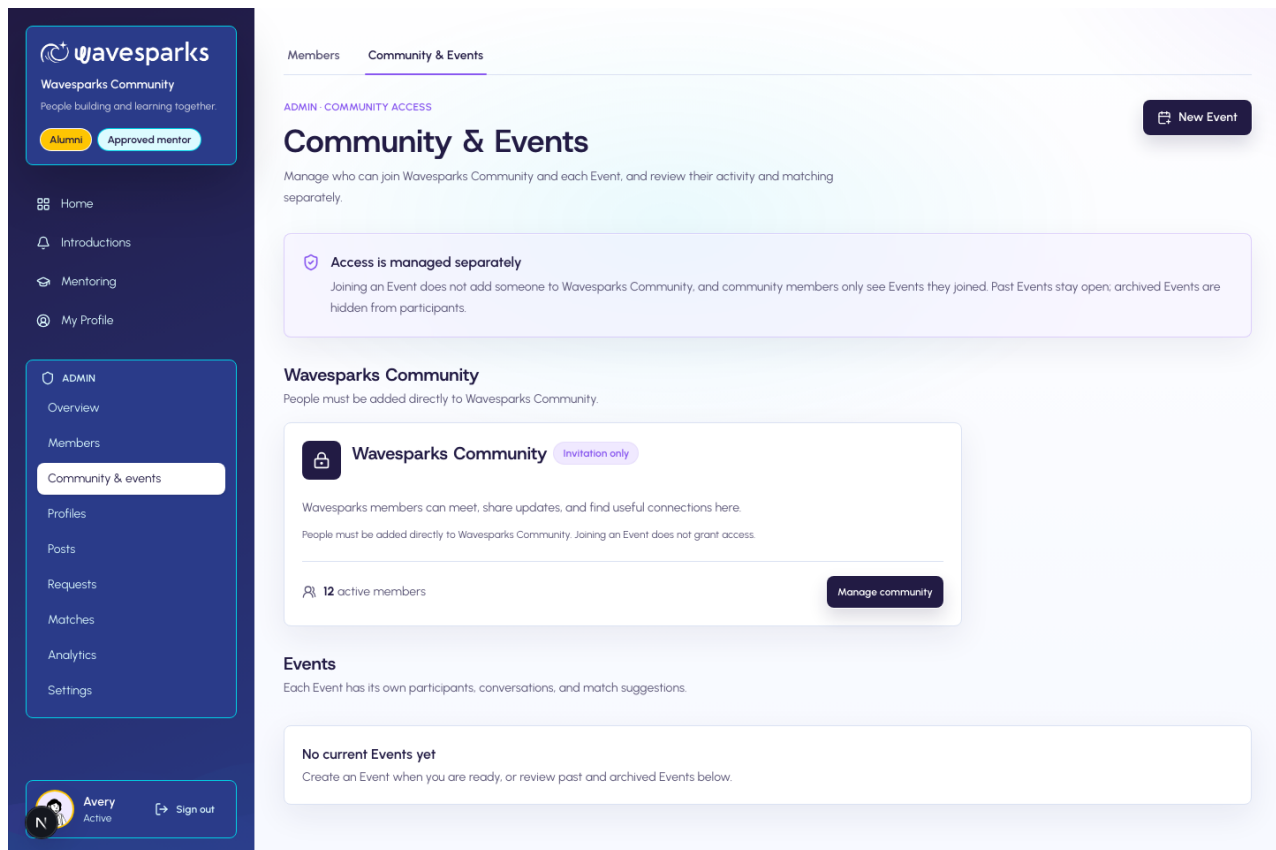
Mentor designation is a service qualification, not an administrative permission. **needs\_review** represents a legacy signal awaiting validation. Only an approved Mentor is included in Mentor discovery and matching. Revoking approval expires pending Mentoring requests and stops new Mentor discovery and matching, while preserving the private Mentor Profile and accepted/declined history.

## 6. Community and Event lifecycle

Each organization has exactly one Main Community and can have multiple Events. Main is permanent, invitation-only, and always active; it cannot be ended or archived.

Event states:

- **draft**: Admin setup only; participants cannot enter.
- **upcoming**: active participants can enter before the start time.
- **active**: standard read, write, and matching behavior.
- **ended**: shown as a Past event, but currently remains readable, writable, and matchable.
- **archived**: hidden from participants; access and matching stop, while data is retained.



Community and Events

## 6.1 Create and publish an Event

1. Create an Event under Community & events.
2. Enter its name, slug, description, tags, schedule, and Matching configuration.
3. Keep it in **draft** while completing content and access checks.
4. Configure the roster through individual invitations or Add participants within the Event.
5. Switch to **upcoming** or **active**, then test access with a real Member account.
6. After the event, use **ended** if interaction should remain available; use **archived** when access must actually stop.

## 6.2 Add to Main Community

From an Event, select eligible participants and choose Add N to Main Community. This action:

- immediately creates an **active** Main entitlement;
- is idempotent for members already in Main;
- preserves the original Event entitlement;

- does not copy posts, follows, matches, feedback, or Introductions;
- reports conflicts row by row.

Do not assume Event participation automatically grants Main access, and do not simulate an "upgrade" by copying data.

## 7. Profile, content, Introduction, and Matching operations

### 7.1 Profiles

Profiles provides access to complete member data, operational contact details, Featured/Stale queues, and CSV export. Before an export, confirm the purpose, the smallest possible recipient list, and a secure storage location. Delete local copies according to the organization's retention policy.

The current export lacks comprehensive central auditing, and part of its authorization logic requires remediation. Fix it before making export a routine operating procedure.

### 7.2 Content moderation

Posts supports:

- Hide/Unhide;
- Feature/Unfeature;
- Lock/Unlock comments;
- Archive/Reopen;
- Remove/Restore images, link previews, and comments.

Before acting, record the Post or Comment ID, Space, reason, and operator. There is currently no working Reports queue or non-repudiable Admin Audit Log. Record evidence and decisions for material incidents in an external ticketing system.

### 7.3 Introductions

Requests can be filtered by status, source, and Space. To create a manual Introduction, the Admin must personally have access to that Space, and both candidates must be current participants with complete Profiles. Enter Who is asking, Who should they meet, Purpose, Note, and Suggested first message.

Admins can view contact details, but must not bypass the two parties' consent by disclosing them to third parties. An Introduction never grants access to a new Space.

### 7.4 Matching

**ADMIN · MATCHES**

## Review match suggestions

See the suggestions people receive in Wavesparks Community and each Event.

**Refresh matches**

**All** Strong matches Good matches Co-founder Collaborator Mentor

**MENTOR**

Wavesparks Community **Good match**

**Kai and Marcus**

Marcus's experience may fit what you are looking for: their capabilities complement the needs you shared; you are working in relevant problem areas. Marcus offers GTM, matching your need for GTM. Relevant signals include fintech, B2B SaaS, sales.

**MENTOR**

Wavesparks Community **Worth exploring**

**Marcus and Kai**

Kai's experience may fit what you are looking for: their capabilities complement the needs you shared; you are working in relevant problem areas. Kai offers operations, matching your need for operations. Relevant signals include fintech, B2B SaaS, operations.

**MENTOR**

Wavesparks Community **Worth exploring**

**Tom and Kai**

Kai's experience may fit what you are looking for: their capabilities complement the needs you shared; you are working in relevant problem areas. Kai offers operations, matching your need for operations. Relevant signals include B2B SaaS, operations, GTM.

**COLLABORATOR**

Wavesparks Community **Worth exploring**

**Kai and Tom**

You and Tom may be able to help each other: your availability and commitment look compatible; the location or time-zone fit looks practical. Tom offers GTM, matching your need for GTM; you offer operations, matching their need for operations. Relevant signals include B2B SaaS, GTM, operations.

### Matching management

Matching runs independently within each Space. A candidate must be connected, have active Space access, have a complete Profile and Space intent, and be opted in. An **ended** Event continues matching; an **archived** Event stops.

The Admin interface can:

- review Strong/Good matches and each Match type;
- create or update Match types, with no more than 12 enabled types;
- configure direction, minimum score, and six weights that total 100;
- trigger a full refresh;
- review anonymous Helpful/Not relevant feedback and recent runs.

Record the current configuration before changing it and validate changes in development or with isolated data first. After a production refresh, sample across Spaces, match types, Mentor gates, hidden items, and dismissed items. Fit index is a measure of match relevance, not a success probability or reputation score.

## 8. System architecture and systems of record

| Area                                                                     | System of record | Notes                                                                                                             |
|--------------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------|
| Code and deployment                                                      | GitHub + Vercel  | There is currently no GitHub Actions gate; delivery primarily depends on Vercel Git integration and manual checks |
| Identity, credentials, Sessions, identity invitations                    | Clerk            | Clerk Organizations are not used                                                                                  |
| Organization, roles, invitation authorization, Spaces, content, matching | Neon/Postgres    | Sole authority for application authorization                                                                      |
| Standard product email                                                   | Resend           | Invitation email does not use Resend                                                                              |
| Avatars, logos, post media                                               | Vercel Blob      | Avatars/logos are public; post media is private                                                                   |
| Semantic embeddings                                                      | OpenAI           | text-embedding-3-large, 1,024 dimensions; failures use a deterministic local fallback                             |

The application region is fixed to `sin1` in `vercel.json`. The Neon region should be selected nearby, but the repository cannot prove which region is actually selected in the vendor console.

## 9. Environment variable management

| Variable                          | Purpose                                                | Production requirement                                                                          |
|-----------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| NEXT_PUBLIC_APP_URL               | Canonical application URL and invitation return target | HTTPS community application domain, not the marketing site                                      |
| NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY | Browser-side Clerk key                                 | Starts with <code>pk_live_</code>                                                               |
| CLERK_SECRET_KEY                  | Server-side Clerk key                                  | Secret; never expose publicly                                                                   |
| CLERK_JWT_KEY                     | Additional Clerk JWT verification                      | Configure for the deployment architecture                                                       |
| CLERK_WEBHOOK_SIGNING_SECRET      | Webhook signature verification                         | Starts with <code>whsec_</code>                                                                 |
| NEXT_PUBLIC_CLERK_*_URL           | Sign-in and sign-up paths                              | <code>/org/wavesparks/...</code> or an HTTPS URL                                                |
| DATABASE_URL                      | Neon Postgres connection                               | Not localhost; isolated by environment                                                          |
| SPACE_SCOPED_READS_ENABLED        | Space-isolation switch                                 | Must explicitly be true; otherwise the application fails closed                                 |
| OPENAI_API_KEY                    | Matching embeddings                                    | Secret                                                                                          |
| RESEND_API_KEY                    | Product notification email                             | Domain-scoped Sending access                                                                    |
| RESEND_FROM_EMAIL                 | Sender address                                         | Must use a verified domain                                                                      |
| CRON_SECRET                       | Cron Bearer authentication                             | Random; at least 32 characters recommended                                                      |
| BLOB_READ_WRITE_TOKEN             | Vercel Blob                                            | Secret; media upload is unavailable if absent                                                   |
| WAVESPAK_ADMIN_EMAILS             | Bootstrap target email addresses                       | The current first-Admin flow has a P0; do not treat these as usable accounts before remediation |
| E2E_CLERK_*                       | Clerk E2E test accounts                                | Not equivalent to the local E2E bypass                                                          |

Never set `E2E_LOCAL_AUTH_ENABLED` or `E2E_LOCAL_AUTH_SECRET` in production. Development, Preview, and Production must use separate Clerk, Neon, and other secrets.

A Vercel environment-variable change affects only later Deployments. Redeploy after every change, then verify that the running instance uses the new value. Never place real values in Git, README files, PDFs, terminal recordings, or pull requests.

## 10. Release and database migration runbook

### 10.1 Before release

1. Resolve the P0/P1 blockers in Section 2 and complete code review.
2. Confirm the Git branch, target Vercel Project, and Production Domain.
3. Establish a recoverable point or branch in Neon, and record its time and owner.
4. Verify that Vercel Production environment variables are complete and contain no Preview or Development credentials.
5. Run the following locally or in controlled CI:

BASH

```
pnpm install --frozen-lockfile
pnpm env:audit
pnpm readiness:prod -- --env-only
pnpm typecheck
pnpm lint
pnpm test
pnpm build
```

`env:audit` requires the development database name to be `wavespark_dev`, requires Clerk test keys in development and live keys in production, and verifies that Development and Production do not point to the same database.

### 10.2 Migration

Start with a dry run:

BASH

```
pnpm db:migrate -- --environment=production
```

After confirming the displayed host, database, and recovery point, an authorized operator runs:

BASH

```
pnpm db:migrate -- --environment=production --apply --confirm-production
pnpm readiness:prod
```

Migration creates the `vector` extension and applies Drizzle migrations. A Production write requires both `--apply` and `--confirm-production`. The full Readiness check also audits exactly one Main Space per organization, duplicate and orphaned relationships, and the integrity of Space-scoped data.

## 10.3 Deployment and verification

1. Allow Vercel to create a Preview Deployment.
2. Test sign-in, Space boundaries, and critical features using isolated Preview data. Preview must not connect to the Production `DATABASE_URL`.
3. Deploy or Promote to Production.
4. Check the home page, Admin interface, real Clerk sign-in, and invited-user acceptance flow.
5. Sample Feed, People, Profile, media, Introduction, and Matching.
6. Inspect Vercel Runtime Logs for 4xx/5xx, webhook, email, and Blob errors.
7. Check both Cron paths and their latest results.

Rolling back the application does not roll back the Neon schema. For releases containing incompatible database changes, use backward-compatible expansion/migration, deploy schema before code, verify, and remove old fields only afterward. Never treat Vercel Rollback as database recovery.

## 11. Vercel operations and maintenance

### 11.1 Current responsibilities

- Host the Next.js 16 application in region `sin1`;
- Manage Production, Preview, and Development environment variables;
- Execute Cron Jobs;
- Provide Runtime Logs and Observability;
- Host Vercel Blob.

Cron schedules use UTC:

- `0 8 * * * -> /api/internal/matches/recompute`, 16:00 Singapore time;
- `30 8 * * * -> /api/internal/post-media/cleanup`, 16:30 Singapore time.

Both use `CRON_SECRET` Bearer authentication and should execute only in Production.

### 11.2 Daily checks

- Deployments: confirm that Production points to the expected commit and the build succeeded.
- Runtime Logs: filter by `requestPath` to inspect Cron, webhook, email, Blob, and 5xx activity.
- Cron Jobs: review recent HTTP status codes, duration, and failures.
- Usage/Spend: review Functions, bandwidth, Blob storage, and request volume.
- Alerts: enable appropriate 5xx, usage, and Spend Management alerts.
- Observability: configure a Log Drain according to retention requirements; the current repository has no persistent alerting implementation.

### 11.3 Environment variables and secret rotation

Use this general sequence:

1. Create a new secret or key at the third-party provider, leaving the old value temporarily valid.

2. Update the correct Environment scope in Vercel.
3. Redeploy.
4. Confirm with real requests and logs that the new value is active.
5. Revoke the old value and verify again.

Do not update Vercel and wait for existing instances to change automatically. Never place a sensitive secret in a `NEXT_PUBLIC_*` variable.

## 11.4 Rollback

Use Vercel to Rollback to or Promote a known-good Deployment, but always:

- inspect the environment-variable version captured by that Deployment;
- confirm compatibility with the current Neon schema;
- check Cron Jobs independently after rollback;
- record incident time, commit, impact, and recovery outcome.

Official documentation:

- <https://vercel.com/docs/environment-variables>
- <https://vercel.com/docs/environment-variables/rotating-secrets>
- <https://vercel.com/docs/cron-jobs/manage-cron-jobs>
- <https://vercel.com/docs/logs/runtime>
- <https://vercel.com/docs/deployments/rollback-production-deployment>
- <https://vercel.com/docs/alerts>
- <https://vercel.com/docs/spend-management>
- <https://vercel.com/docs/regions>
- <https://vercel.com/docs/vercel-blob>

## 12. Clerk operations and maintenance

### 12.1 Current boundary

Clerk manages Users, verified email addresses, credentials, Sessions, and Application Invitations. WaveSparks does not use Clerk Organizations; organizations, roles, memberships, and Space entitlements are stored in Neon.

Normal invitations must originate in WaveSparks Admin. Creating an Invitation directly in Clerk Dashboard establishes identity only and does not create the local authorization chain.

Webhook endpoint: `/api/webhooks/clerk`. Subscribe to:

- `user.created`;
- `user.updated`;
- `user.deleted`.

The webhook verifies signatures and deduplicates by `svix-id`. Created/Updated only synchronize an already-bound identity. Deleted anonymizes the local user. Clerk Organization events never grant permissions.



## 12.2 Initial configuration

- Use Clerk live keys in Production and keep the instance separate from Development test keys.
- Configure the Production domain, DNS, and allowed Redirect URLs.
- Keep Restricted sign-up enabled so sign-up opens only in a valid invitation context.
- Create a public HTTPS webhook endpoint and select the three User events.
- Store the Signing Secret in Vercel Production and Redeploy.
- Use a test account to verify invitation, acceptance, update, deletion, and failed-event replay.
- Do not enable Clerk Organization selection for this project; retain the application's own organization model.

## 12.3 Daily operations and incident response

- Invitation not received: inspect Clerk Invitations, Application Logs, the destination address, and service status.
- Acceptance failure: verify Invitation state, seven-day validity, verified email address, and Clerk User ID binding.
- Webhook failure: inspect Attempts, response status, and matching Vercel logs; fix the cause and Replay.
- Identity compromise: suspend in WaveSparks and revoke Sessions or block the user in Clerk.
- Deletion not synchronized: confirm delivery of `user.deleted` and the correct signature secret, then inspect local anonymization.

Webhooks are asynchronous and eventually consistent. Keep the handler idempotent; never treat it as an immediate transaction inside a user request.

## 12.4 Zero-downtime rotation

Clerk Secret Key:

1. Create a second active key.
2. Update Vercel Production and Redeploy.
3. Confirm the new key through Clerk last-used data or Application Logs and a real request.
4. Delete the old key.

Webhook Secret:

1. Create a new webhook endpoint and Secret.
2. Update Vercel and Redeploy.
3. Send a test event and verify signature handling and deduplication.
4. Delete the old endpoint.

Official documentation:

- <https://clerk.com/docs/guides/development/deployment/production>
- <https://clerk.com/docs/guides/secure/rotate-api-keys>
- <https://clerk.com/docs/guides/users/inviting>
- <https://clerk.com/docs/guides/secure/restricting-access>
- <https://clerk.com/docs/guides/development/webhooks/overview>

- <https://clerk.com/docs/guides/development/webhooks/syncing>
- <https://clerk.com/docs/guides/dashboard/logs/application-logs>
- <https://clerk.com/docs/guides/secure/session-options>

## 13. Neon operations and maintenance

### 13.1 Current boundary

The codebase reads only `DATABASE_URL`. Regular queries use the Neon HTTP driver; transactions and migrations use a single `postgres-js` connection. Migration requires the `vector` extension.

Runtime and Migration currently share one database credential; least-privilege roles have not been separated. The repository contains no automation for Neon Branches, Snapshots, recovery, or alerting.

### 13.2 Connection selection

Prefer a pooled endpoint containing `-pooler` for serverless runtime traffic to reduce concurrent connection pressure. Use a direct endpoint for migrations, `pg_dump`, or tools that require direct-connection semantics. Before changing the connection, review current Neon guidance and the project's connection mode.

### 13.3 Migration, branching, and recovery

- Create a point-in-time branch or snapshot before every Production migration, and record the recovery point.
- Validate schema diff, Migration, and application compatibility on an isolated Branch.
- Run a recovery drill on an isolated Branch every quarter and record RTO/RPO.
- Configure the Restore window for the plan; enable protection and Scheduled snapshots for important Production branches where available.
- Preview databases must remain isolated and must never reuse the Production `DATABASE_URL`.
- A Vercel application rollback is not a substitute for Neon PITR.

For recovery, first freeze writes and record the target time. Validate the data and application on an isolated Branch before choosing whether to switch the connection or apply a forward fix. Irreversible data changes require joint approval from the database owner and business owner.

### 13.4 Daily monitoring

Review:

- CPU, RAM, database size, and Compute active time;
- Client/Server connections and Pooler metrics;
- cache hit rate, query latency, deadlocks, and errors;
- Branch and Storage growth and the Restore window;
- latency between the Vercel and Neon regions;
- Space data-integrity results from Readiness.

## 13.5 Credential rotation

Prefer creating a new Database role, copying only the necessary Grants, updating Vercel, Redeploying and verifying, and then revoking the old Role. A direct password reset immediately invalidates old connections. Runtime and migration currently share a credential; genuine least-privilege separation requires code and deployment configuration changes.

Official documentation:

- <https://neon.com/docs/manage/projects>
- <https://neon.com/docs/guides/branching-intro>
- <https://neon.com/docs/connect/connection-pooling>
- <https://neon.com/docs/guides/schema-diff>
- <https://neon.com/docs/manage/endpoints/>
- <https://neon.com/docs/changelog>

## 14. Resend operations and maintenance

### 14.1 Current boundary

Resend sends only standard product notifications, such as Introduction requested, accepted, or declined. Clerk sends member invitation email.

The current implementation sends asynchronously through Next.js `after()`:

- when Resend is not configured, the application only records `provider unconfigured`; in-app notifications continue;
- a send failure is written only to the Vercel console;
- there is no webhook, delivery/bounce/complaint status table, outbound audit, or Admin retry interface;
- the lower-level sender supports an Idempotency key, but current product calls do not pass one.

Therefore, Resend Dashboard and Logs are the primary sources for current email-delivery operations.

### 14.2 Domain and API Key

- Use a dedicated sending subdomain to isolate reputation.
- Configure and verify SPF and DKIM.
- Start DMARC with `p=none` for observation, then tighten it gradually according to organizational policy.
- Give the API Key Domain-scoped Sending access, not Full access.
- `RESEND_FROM_EMAIL` must belong to a verified domain.

### 14.3 Daily checks and response

Check `failed`, `bounced`, `complained`, `suppressed`, and recent volume each day:

- Failed: correlate Vercel logs to identify configuration, quota, or destination problems.
- Bounced: correct the address before resending; do not repeatedly send to a hard bounce.
- Complained: stop sending and inspect the consent basis.

- Suppressed: resolve the root cause before removing suppression; do not repeatedly override it.
- Invitation not received: investigate Clerk, not Resend.

If auditable delivery is required, add a Resend webhook: verify the signature, deduplicate by `svix-id`, tolerate at-least-once delivery and out-of-order events, and retain only the minimum required state. Product emails that may be retried should actually pass an Idempotency key. Resend's deduplication window is 24 hours.

## 14.4 Zero-downtime rotation

1. Create a new Domain-scoped Sending Key.
2. Update the Vercel Production environment variable.
3. Redeploy and send a controlled test notification.
4. Verify the Key and message in Resend Logs.
5. Delete the old Key and confirm delivery again.

Official documentation:

- <https://resend.com/docs/dashboard/api-keys/introduction>
- <https://resend.com/docs/knowledge-base/how-to-handle-api-keys>
- <https://resend.com/docs/dashboard/domains/introduction>
- <https://resend.com/docs/dashboard/domains/dmarc>
- <https://resend.com/docs/dashboard/emails/introduction>
- <https://resend.com/docs/dashboard/emails/email-suppressions>
- <https://resend.com/docs/webhooks/introduction>
- <https://resend.com/docs/webhooks/verify-webhooks-requests>
- <https://resend.com/docs/dashboard/emails/idempotency-keys>

## 15. Vercel Blob and media maintenance

- Avatar and Organization Logo use public Blob URLs; never upload sensitive images.
- Post images and link previews use private Blob and are read through application authorization.
- A post can contain up to four images, each no larger than 5 MB, in JPG, PNG, or WebP format; the longest edge is processed down to 2,400 px.
- Media upload is unavailable when `BLOB_READ_WRITE_TOKEN` is absent; text features remain available.
- A daily cleanup Cron handles orphaned media; failures currently exist only in Runtime Logs.

Review Blob usage and Cleanup Cron daily. When content is taken down, confirm that both the database moderation state and Blob lifecycle satisfy the retention policy. Rotate the Token in this order: new value -> Vercel -> Redeploy -> upload/read verification -> revoke old value.

## 16. Security incident and offboarding runbook

### 16.1 Member account incident

1. Globally Suspend the member in Members and record the reason.

2. If identity compromise is suspected, revoke all Sessions or block the user in Clerk.
3. Inspect recent Admin, Runtime, and Application Logs. Because a complete application audit trail does not exist, correlate with an external incident ticket.
4. Address affected Spaces, Invitations, content, and Introductions.
5. Remove the Clerk and application restrictions separately only after recovery criteria are met.

## 16.2 Administrator departure

1. First confirm that at least one other administrator can sign in.
2. Revoke Admin permission or deprovision the account in WaveSparks.
3. Revoke Sessions or disable the identity in Clerk.
4. Remove access from GitHub, Vercel, Neon, Resend, and Clerk consoles.
5. Rotate any Secret the person may have accessed, and review recent Deployments, Exports, and changes.
6. Record completion time and the second reviewer in the external audit record.

## 16.3 User deletion

After the Clerk `user.deleted` webhook succeeds, the local Profile and contact data are anonymized, the account is deactivated, related follows, saves, matches, and notifications are deleted, and pending Introductions expire. Historical posts and comments remain attributed to Former member.

Explain the retention policy before deletion. If the Clerk identity is deleted while the webhook fails, local anonymization does not complete automatically; fix the failure and Replay the event.

# 17. Recurring maintenance checklist

## Daily

- Vercel Production Deployment, 5xx responses, and Runtime Logs;
- HTTP results and path logs for both Cron Jobs;
- Clerk Invitation and webhook failures;
- Resend failed, bounced, complained, and suppressed messages;
- Neon errors, connection anomalies, and capacity anomalies;
- pending Invitations, Introductions, and content operations queues.

## Weekly

- Sample Space access isolation and `ended/archived` states.
- Review member-import failures, Stale Profiles, Matching runs, and feedback.
- Review Blob, Functions, Database, and Email usage.
- Reconcile the Admin list and third-party console access lists.

## Monthly

- Run the Production Readiness data audit.

- Review dependency updates, security advisories, alerts, and Spend.
- Test invited sign-up, webhook, email, media, and critical Matching paths.
- Remove no-longer-needed Preview Deployments, Branches, and exported files.

## Quarterly

- Run an isolated Neon recovery drill.
- Rehearse rotation of Clerk, Resend, Blob, Cron, and other Secrets.
- Run tabletop exercises for Admin departure and security incidents.
- Review RTO/RPO, data retention, and vendor plan limits.

## 18. Troubleshooting quick reference

| Symptom                                           | Check first                                                                                          |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Bootstrap Admin cannot sign in                    | Current P0 first-Admin connection gap; do not use an email fallback, deploy the supported fix first  |
| Connected user sees only My Spaces                | Verify an active entitlement for the target Space                                                    |
| Event participant cannot see Main                 | Expected behavior; explicitly Add to Main                                                            |
| User can still post in an Ended Event             | Current design; Archive the Event to stop access                                                     |
| Admin is absent from People/Matching              | Verify that the Admin is explicitly added to that Space                                              |
| User can read but cannot interact                 | Check all seven Profile completion requirements                                                      |
| No matches                                        | Check Profile, Space intent, opt-in, configuration, candidate count, and lifecycle                   |
| Invitation not received                           | Check Clerk Invitation/Application Logs, not Resend                                                  |
| Product notification not received                 | Check in-app notification, Resend Logs, Vercel Runtime Logs, and target Space access                 |
| Webhook does not synchronize                      | Check Clerk Attempts/Replay, signing Secret, Vercel logs, and the idempotency record                 |
| Media upload fails                                | Check Blob Token, size/type, usage, and Runtime Logs                                                 |
| Cron fails                                        | Check Vercel Cron, CRON_SECRET, request-path logs, and database/OpenAI/Blob dependencies             |
| Database errors remain after application rollback | Verify old-code compatibility with the current Neon schema; use PITR or a forward fix when necessary |

## 19. Release acceptance checklist

- The first Admin is connected through the supported invitation and binding flow; the P0 is closed by code and tests.
- Email-authorized Route Handlers and the Preview Accounts P1 are remediated.
- Development, Preview, and Production use completely separate Clerk, Neon, Resend, and Blob resources.
- `SPACE_SCOPED_READS_ENABLED=true`, and no E2E bypass exists in Production.

- Environment audit, Production Readiness, typecheck, lint, tests, and build all pass.
- A verifiable recovery point exists before Migration, and the post-Migration data audit passes.
- Clerk live keys, Restricted sign-up, Redirect configuration, and all three User webhooks are correct.
- The Resend domain passes SPF/DKIM, and the Sending Key has minimum permissions.
- Both Vercel Cron Jobs, Runtime Logs, Alerts, and Spend controls have been reviewed.
- Event-only, Main-only, multi-Event, Admin, Mentor, Suspended, and Archived scenarios pass.
- Feed, People, Posts, Notifications, Introductions, and Matches show no cross-Space leakage.
- Vercel rollback and Neon restore have each been rehearsed independently.
- The administrator, console Owner, and emergency-contact lists have been reviewed by two people.